PACE: Full Project Delegation & Workflow Specification

Lead Engineer & Engineering Team

July 2026

1. Collaboration Strategy: Compute vs. Logic

The PACE (Pipelined Actor-Critic Ensemble) architecture naturally splits into two distinct engineering domains. To optimize development velocity, prevent Git merge conflicts, and accommodate hardware disparities, the complete project workload is strictly divided as follows:

- **Stream A (High-Compute / GPU):** Assigned to the Lead Engineer. Requires dedicated GPU resources (NVIDIA RTX 4060, 8GB VRAM). Focuses strictly on PyTorch mechanisms, Transformer architectures, mathematical optimization, and the VRAM state machine.
- **Stream B (Low-Compute / CPU):** Assigned to the Collaborator. Requires standard CPU/RAM. Focuses on data engineering, algorithmic mutation logic, scraping, validation sandboxes, and the full-stack FastAPI/UI backend.

2. Stream A: Lead AI Engineer (GPU-Bound Tasks)

Assigned to: Project Lead (High-Compute Rig)

Your domain is the core artificial intelligence engine. You will write all PyTorch logic, manage the GPU memory budget, and train the models for all three Masteries.

Global Architecture Responsibilities

- **PyTorch Data Pipelines:** Constructing `Dataset` and `DataLoader` classes to ingest Parquet files and yield batched, tokenized tensor matrices for all masteries.
- **Transformer Architecture:** Defining the Actor (Decoder-only) and Critic (Encoder-only) model classes, including self-attention mechanisms and forward passes.

- **The VRAM State Machine:** Engineering the hot-swap protocol to load and unload models sequentially, utilizing `del` and `torch.cuda.empty_cache()` to enforce the 8GB memory ceiling during live inference.

Mastery-Specific Training Operations

- **Coding Mastery:** Training the Coding Actor to generate Python and the Coding Critic to perform AST-level binary classification (Clean vs. Buggy).
- **Literacy Mastery:** Training the Literacy Actor for summarization/extraction, and the Literacy Critic to perform NLI (Natural Language Inference) entailment scoring to penalize hallucinations.
- **Research Mastery:** Fine-tuning the Research Actor to synthesize abstracts into reviews, and training the Research Critic to verify citation IDs and factual accuracy.

3. Stream B: Data & Backend Engineer (CPU-Bound Tasks)

Assigned to: Collaborator (Standard Rig)

Your domain is the infrastructure, data pipelines, and validation engines. You build the environments that the AI interacts with, generate the training data, and serve the final application.

Mastery 1: Coding (Execution & Logic)

- **Synthetic Bug Engine:** Writing Python scripts using Abstract Syntax Tree (AST) parsing to automatically inject operator flips, logic deletions, and off-by-one errors into clean code.
- **Sandbox Verification Engine:** Building the secure Python `subprocess` execution environment with strict timeouts to run LLM-generated code against unit tests safely.

Mastery 2: Literacy (Data & Text Processing)

- **Corpus Ingestion:** Writing web scrapers (e.g., BeautifulSoup, Scrapy) or API connectors to pull clean articles, internal wikis, or books.
- **Text Chunking & Token Prep:** Building the text chunking logic (e.g., overlapping sliding windows) to ensure documents fit within the Transformer's maximum token context limit.

- **Heuristic Evaluation:** Setting up standard NLP evaluation metrics (ROUGE, BLEU) as baseline comparisons against our custom Critic models.

Mastery 3: Research (Parsing & Vectorization)

- **PDF Abstract Parsing:** Utilizing tools like PyMuPDF or Grobid to extract text, titles, and citation IDs cleanly from academic PDF papers.
- **Citation Regex Verification:** Writing strict Regular Expressions to parse AI-generated literature reviews and map the generated citation brackets (e.g., *[Author, 2023]*) back to the input abstracts.

Global Full-Stack Infrastructure

- **FastAPI Microservice:** Building the async Python backend. Creating strict Pydantic models for incoming user prompts and outgoing AI responses.
- **User Interface (UI):** Developing the frontend dashboard (using Streamlit, Gradio, or React) where users can select a Mastery, input a prompt, and view the Actor-Critic pipeline process in real-time.
- **Dockerization:** Containerizing the FastAPI backend, the Sandbox verification environment, and the UI for seamless one-click deployment.

4. Integration & Git Workflow

To guarantee zero merge conflicts and seamless integration, both engineers will adhere to strict repository boundaries:

- **Stream A Territory:** `masteries/*/training/` (Contains `train.py`, `dataset.py`, `models.py`).
- **Stream B Territory:** `masteries/*/data/`, `core/sandbox/`, `api/`, and `ui/`.
- **The API Bridge:** Stream A will wrap the final, trained PyTorch models inside simple inference functions. Stream B will import those functions into the FastAPI routes. Stream B does *not* need to understand PyTorch to pass strings into these functions and receive strings back.

Document automatically generated for the PACE Architecture Planning Phase.